

一文读懂高德地图AJX容器架构原理2

一、前言

AJX容器至今已经走过了六个年头，服务了数亿中国用户。高德AJX支撑团队低调务实，扎扎实实做技术，打造了一套底层稳定可靠、性能媲美原生，同时兼具优秀开发体验的跨端方案。

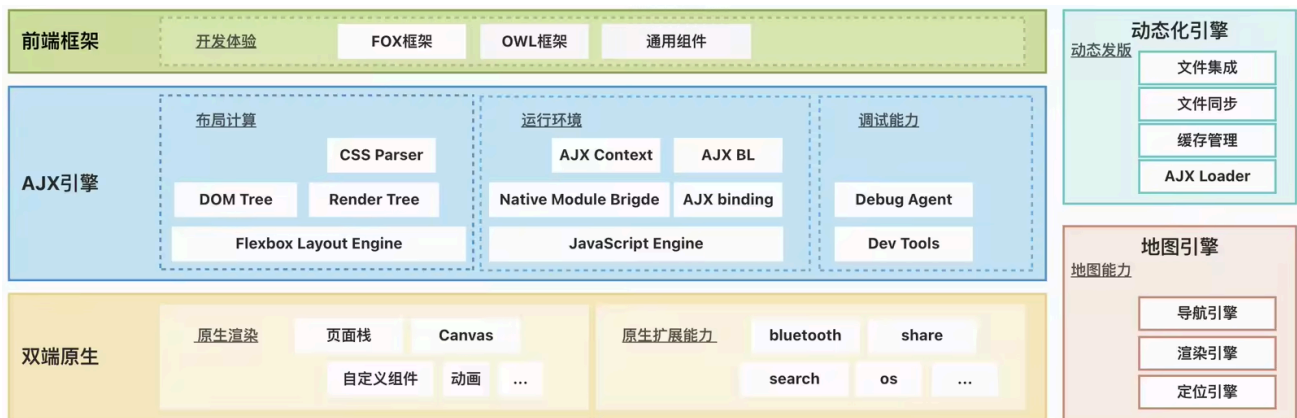
AJX自诞生之初就开始接受质疑，业界已经有方案了，为什么还要自研？决定自研之后接下来的两年里铺开大量业务，又开始在性能、稳定性和开发体验上接受挑战：

- 因为某个页面的复杂度太高性能过不去，当时我们支持了预加载能力；
- 为了增强首屏用户体验，我们在AJX支持了骨架屏；
- 为了解决小米某款x86平板的集中崩溃，专门采购了这款平板分析问题，并且在商店提供了独立的安装包；
- 为了支持打车，我们参考Android的Service机制提供了具有独立线程的AJXService；
- 为了开发体验，我们基于VS Code开发了AJXIDE, 并且支持了js调试、DOM调试；
-

正因为这些创新点的不断积累，才沉淀出AJX框架今天的样子。从今天往回看，即使我们采用了当时业界已有的方案，为了适应我们的业务，也必然会走出一条不同的路。而且跨端方案也必须需要及时，有力的支撑，遇到问题时能随时处理，而不能给社区提一个issue等修复。关于AJX更多的演进历史可以在这里了解：《[揭秘高德地图动态化跨端容器--AJX](#)》。

二、从架构图说起

业界普遍认为对旧要素的新组合即是创新，AJX是一种创新。就像Safari、Chrome等浏览器在性能、功能设置、隐私工具亦或调试体验存在差别一样，对各种要素的不同组合方式，表现出了不同的跨端框架。[React Native](#)是在有了React的基础上推导出跨端方案。而AJX充分利用C++跨端、高性能的特点，研发了一系列符合前端标准的，足够日常开发使用的Web APIs子集以及CSS子集，基于此，由前端专家封装出最符合业务实践的框架，可以是React、vue或者自研框架，从而增强开发体验。



2.1 AJX引擎

•AJX引擎使用C++开发，是整个方案的核心，其组成从本质上可以抽象为：**布局计算、运行环境和调试能力**。布局计算和运行环境下边会分别详细介绍，调试能力包括JS断点调试、DOM调试、AJX文件浏览器以及性能排查工具等，后续会有专题文章进行介绍。

2.2 双端原生

- 主要提供“原生渲染”和“原生扩展”能力。AJX引擎会通过接口持续发送渲染指令，双端平台根据指令进行渲染。比方AJX中的“label”组件最终会在Android中对应“TextView”，在iOS中对应“UITextView”实现，双端原生也可以通过新增标签与实际组件实现的映射来灵活扩展自定义组件的。
- “原生扩展”能力进一步赋能了JS代码。AJX团队开创性的完成了“五独”的目标，即独立编译、独立构建、独立发布、独立测试和独立运营。为了完成这一目标，我们进行了AJX容器的API标准化，更多细节可参考《[高德地图AJX容器API标准化演进之路](#)》。

2.3 前端框架

- 使用AJX引擎提供的Web APIs进行前端框架的封装，以支持更好的开发体验。目前在AJX中提供了FOX框架（类React框架）和OWL框架两套。需要特别说明的是，AJX整套方案并不限制前端框架的使用，所以你也可以使用vue进行封装。

2.4 动态化引擎

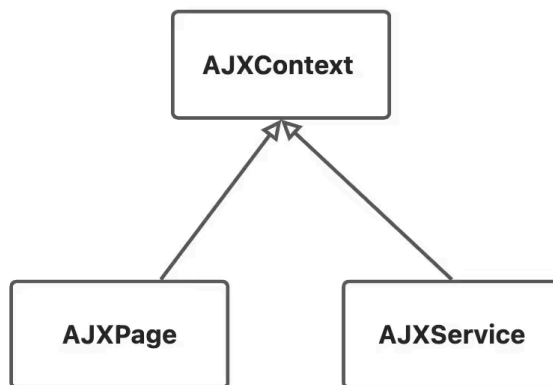
- 动态化引擎是AJX引擎之外的独立模块，提供了动态发版能力，从而使AJX能支持热更新。其组成包括文件集成，文件同步以及缓存管理能力，AJX文件加载器的实现也在这里。更多细节可参考《[高德架构升级：客户端动态框架篇](#)》

2.5 地图引擎

- 地图引擎的核心代码都是C++编写的，AJX引擎层的“AJX BL”模块负责与地图引擎层进行通信并提供对业务的使用接口。

三、详解AJX的运行环境

3.1 AJX上下文



在AJX中有两种上下文，一种是用户可见、可交互的页面（AJXPage），另一种是不可见、限制了UI操作的后台服务（AJXService），AJXService的设计参考了Android原生的Service设计。

3.2 线程模型

•main/Main

○ main/Main线程也被称为UI线程或者主线程，是由Android/iOS原生平台创建的，通常来说所有View的更新都需要在UI线程完成。

• JavaScriptThread

○ 可以认为是AJX的主线程，所有的AJX页面运行在此线程，页面中的布局计算也是在这个线程。

• JsServiceThread

○ AJX Service线程，所有的AJX 服务都在此线程执行。

• AJXIOThread

○ 执行IO操作的独立线程，IO线程不存在JS虚拟机实例，因此不能执行JS代码

• AJXBizEngine

○ 动态引擎的工作线程，用于支持动态化能力。

• 以上线程除UI线程外，均是由AJX引擎在初始化时创建的。其中JavaScriptThread线程和JsServiceThread线程都包含JS虚拟机实例，可以用来并发执行JS脚本。

• AJX引擎中使用了类似Android平台的**消息机制**，消息可分为即时消息和延时消息，比如我们使用定时器1秒之后执行某个任务，就是在消息队列中添加了1秒的延时消息；即时消息并不意味着能马上执行，而是放在消息队列排队执行。

3.3 JavaScript引擎

iOS端由于系统中已经预置了jsc（[JavaScriptCore](#)），所以AJX中直接使用jsc引擎。Android端由于其开放性可自由选择JS引擎，目前AJX在调试包中使用了[v8引擎](#)，其他安装包都是jsc引擎。这样的选择是因为根据我们的实践发现jsc比v8初始化时间更短，更有利于缩短app冷启动时间，而且jsc内存占用更低且线上表现更稳定可靠。v8也有调试能力更好的优点，所以在调试包中仍保留了v8引擎。

3.4 Native Module Bridge

在AJX引擎初始化时Native（Android/iOS）端会向引擎中输入一份JSON格式的Native Module描述信息表。这份信息表中说明了Module名称，以及其对应所有的方法名称。在每个AJX上下文创建时，把表中描述的所有信息动态绑定到natives对象上，比如用于显示提示信息：

▼ 复制代码

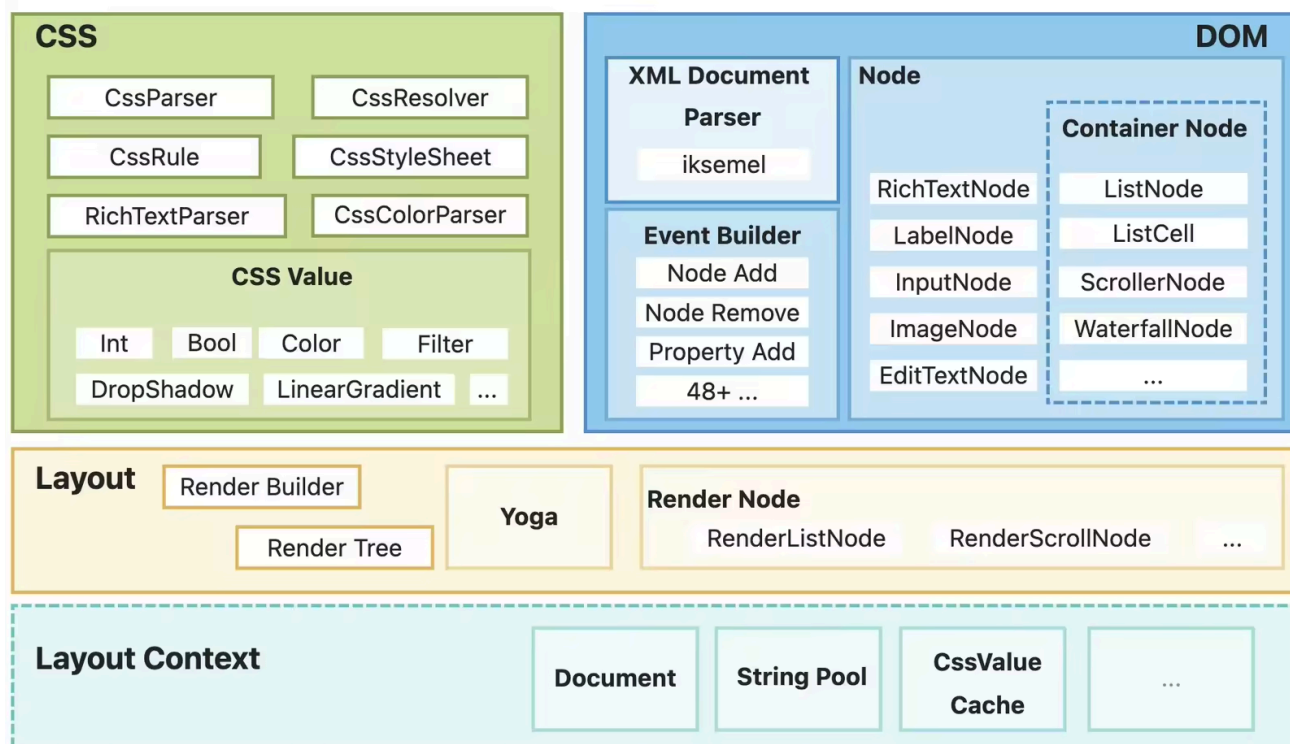
```
natives.app.toast("Hello,world");
```

当Module方法被调用时由AJX引擎做为通道把Module名称、方法名和参数传递给双端原生，双端原生再通过反射的方式进行调用。

四、AJX的布局计算能力

4.1 布局能力架构图

布局能力原本是耦合在AJX运行环境中的模块，后来解耦是一个独立模块，可以从AJX引擎中剥离出来供其他项目使用，整体架构图如下：



4.1.1 CSS

- 样式解析与匹配模块。

4.1.2 DOM

- 文档对象模型，使用了开源工程iksemel进行xml的解析，用于生成DOM树。
- 定义了Node作为节点基类，ContainerNode 继承 Node 用于表示容器节点。
- Event Builder 用于最终生成 UI 事件给双端利用Native原生能力进行渲染，UI 事件根据不同类型附带有不同的数据。

4.1.3 Layout

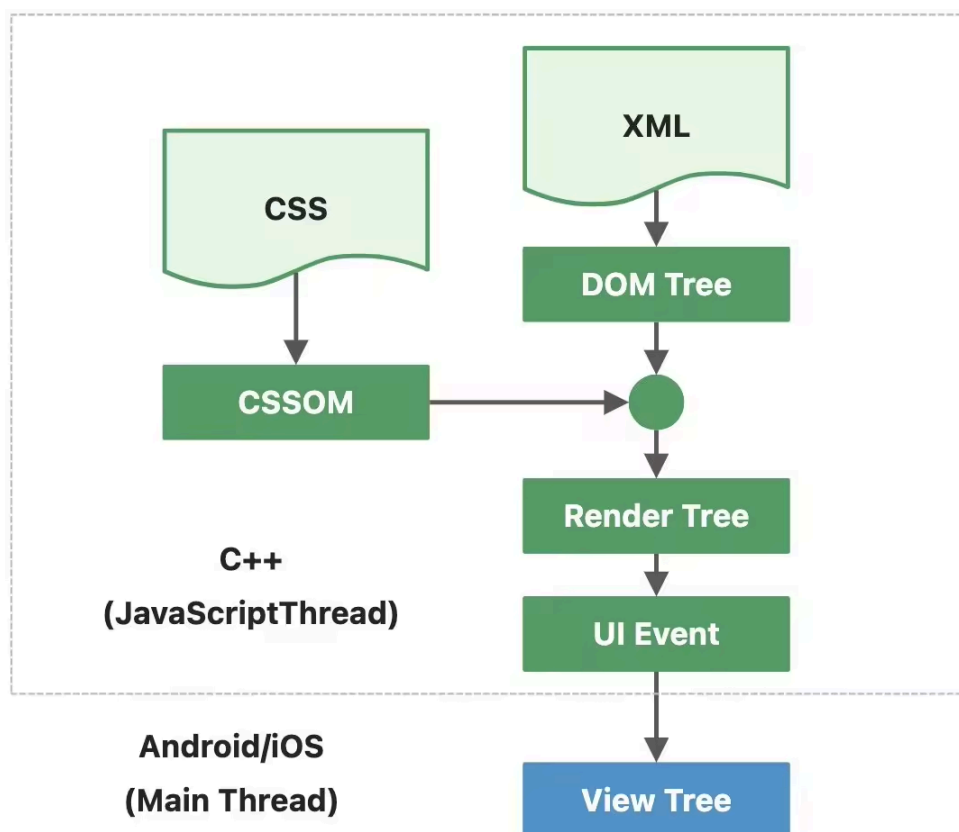
- 布局计算算法使用了与 React Native 一致的开源项目 [facebook/yoga](https://github.com/facebook/yoga)，在接入以及测试的过程中，AJX支撑团队共计提交了 10 个有效 issue 以及一次 PR。

4.1.4 Layout Context

- LayoutContext即布局上下文的概念。每个LayoutContext中的Document实例封装了body根节点和样式表。
- String Pool 属于性能优化点，提供了字符串缓存池，避免了大量的 string 对象的频繁创建和销毁，在 LayoutContext 销毁时统一清空缓存。

4.2 布局计算流程

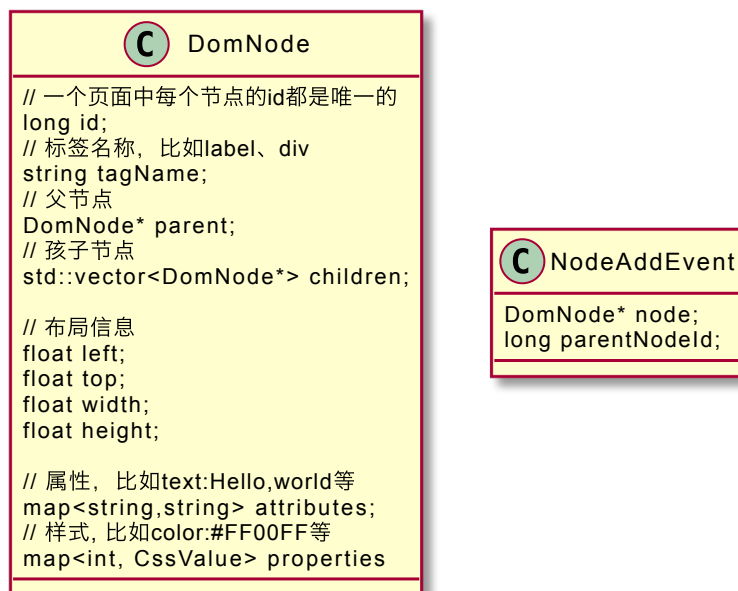
4.2.1 流程图



CSS与XML作为输入，分别生成CSSOM（CSS 对象模型）与DOM Tree，CSSOM 与 DOM 一起构建Render Tree（渲染树），Render Tree经过布局计算后生成一系列的UI Event。

4.2.2 详解UI Event

UI Event是AJX特有的概念，比如在调用 `element.appendChild(aChild)` 之后会生成 `NodeAddEvent` 事件，这个事件中描述要添加的节点详细信息，以及父节点的id，双端原生代码再根据UI Event构建成原生的View进行渲染。下边类图示意了NodeAddEvent所包含的信息，其包括了树结构、布局信息、所有的属性和样式。双端原生根据这些信息生成原生的View，并设置相应的样式。

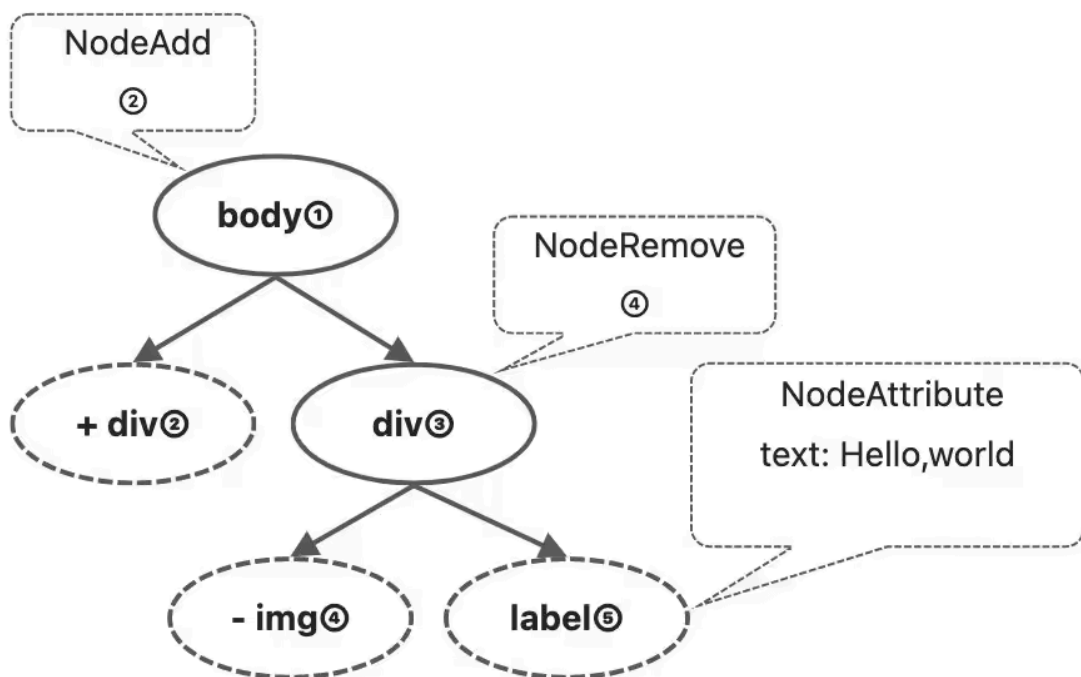


在前端框架里经常会大篇幅的讨论虚拟DOM DIFF算法，在AJX引擎里这一切变得非常简单。所有对DOM树的操作都会以UI Event的形式记录到对应的节点上，最后按DOM树深度遍历并收集UI Event输出到双端原生层。

可以用苹果树来比喻这个过程，苹果树会长满苹果，成熟后统一采摘装车发往果汁厂。这里有个关键信息，并不是苹果树上长一个苹果就立即装车发出去，而是在一个阶段里，生长好一批苹果后，再收集装车，在AJX引擎里定义这个阶段为**执行单元**。在AJX中的页面生命周期、定时器任务、来自Native Module的回调都是一个独立的执行单元，每个执行单元结束都会根据dirty染色判断是否需要重建样式，布局计算。

下图演示了在一个假定的DOM树上进行以下操作：

- body①添加新节点div②
- div③移除节点img④
- label⑤设置text为“Hello,world”



以上操作每个步骤都会生成UI Event，执行完毕后，引擎计算布局，然后从body节点开始深度遍历，收集所有UI Event，交给双端原生进行渲染。在AJX中，事件类型包括了节点增删以及替换，属性样式添加及移除，节点位置尺寸变化，动画播放与暂停等共51类事件。

五、结语

本文从本质上介绍了AJX跨端框架，所以会比较抽象，后续我们会不断推出具体模块详细实现的文章，并添加链接，欢迎收藏本文。如果有想更多了解的内容，也欢迎留言交流，AJX支撑团队有问必答！